

27/1/26

Experiment No. 1

1) count strings using STL.

#include <iostream>

#include <string>

#include <sstream>

using namespace std;

```
int counts (string str) {  
    stringstream ss (str);
```

```
    string word;
```

```
    int count = 0;
```

```
    while (ss >> word) {
```

```
        count ++;
```

```
    }
```

```
    return count;
```

```
}
```

```
int main() {
```

```
    string s = "hello world";
```

```
    cout << counts(s) << endl;
```

```
    return 0;
```

```
}
```


2) Transpose of matrix

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int r, c;
    cout << "Enter rows and columns";
    cin >> r >> c;
    vector<vector<int>>> mat(r, vector<int>(c));
    vector<vector<int>>> trans(c, vector<int>(r));
    cout << "Enter matrix elements: \n";
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cin >> mat[i][j];
        }
    }
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            trans[j][i] = mat[i][j];
        }
    }
    cout << "Transpose matrix: \n";
    for (int i = 0; i < c; i++) {
        for (int j = 0; j < r; j++) {
            cout << trans[i][j] << " ";
        }
    }
    cout << endl;
}
```

iii) Fibonacci Series

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n = 10;
    vector<long long> fib = {0, 1};
    for (int i = 2; i < n; i++)
        fib.push_back(fib[i-1] + fib[i-2]);
    for (auto x: fib)
        cout << x << " ";
}
```

iv) Base / Power

```
#include <iostream>
using namespace std;
int main() {
    double m, n;
    double result;
    cout << "Enter your base and power";
    cin >> m >> n;
    result = pow(m, n);
    cout << result << endl;
}
```


5) Find Time complexity

```
int try_fun(int n) {
    int cnt = 0;
    for (int k = 0; k < n; k++)
        for (int p = k; p > 0; p--)
            cnt = cnt + 1;
    return cnt;
}
```

Time complexity: $O(n^2)$

3/2/26

1) Dry Run

$b = 5$, Binary of 5: 0101, of 4: 0100
5 & 4 result: 0100 (4)

Since $4 \neq 0$, function returns no.
∴ 5 is not a power of 2

2) Dry run

num = 5 pos = 0
5 = 101
pos = 0
1 < 0 = 0001

num = 0101
1 < pos = 00018

0001 = 1

∴ (1) 881 = 1

result = 1

5/2/26

Experiment No. 2

1) To check if a no. is power of 2 or not
→ #include <iostream>

using namespace std;
int main() {

int n;
cout << "Enter a number:";
cin >> n;

if (n > 0 && (n & (n - 1)) == 0)

cout << "The no. is a power of 2";
else

cout << "The no. is NOT a power of 2";

2) Set / Unset

→ #include <iostream>

using namespace std;
int main() {

int num, pos;

cout << "Enter number:";

cin >> num;

cout << "Enter position:";

cin >> pos;

int result = (num & (1 << pos)) && 1;

cout << "Output:" << result << endl;

}

3) Max AND value using bitwise operators.

#include <iostream>
using namespace std;
int main() {

int a[] = {4, 3, 2, 5, 8};
int n = 5;

int maxAND = 0;
int x = 0, y = 0;

for (i = 0; i < 5; i++) {
for (int j = i + 1; j < n; j++) {
int (val > maxAND) {

maxAND = val;
x = a[i];
y = a[j];

}
count << "Max AND value: " << maxAND << endl;
count << "Pair: (" << x << " " << y << ") " << endl;
}

Dry Run: n = 3,
arr = {5, 6, 3}

Pair (arr[i], arr[j])	Binary	max AND
5 & 6	101 & 110 (1000)	4
5 & 3	101 & 011 (1)	4
6 & 3	110 & 011 (2)	4

Highest val → 5 & 6

101 (5)
110 (6)

1000

4) Element occurring odd no. of times.

#include <iostream>
using namespace std;
int main() {

int n;
count << "Enter no. of elements:";
cin >> n;
int arr[n];
int result = 0;

count << "Enter elements:";
for (int i = 0; i < n; i++) {
cin >> arr[i];

result ^= arr[i];
}

count << "The element occurring odd no. of times:
<< result << endl;

Dry Run
 $x^y = x \wedge arr[i]$
 $\therefore \text{result} = 0 \wedge 4$

$= 0 \wedge 0100$
 $= 4$

$4 = 4 \wedge 2$
 $6 = 0110$
 $6 = 6 \wedge 2$
 $= 0100 = 4$
 $4 = 4 \wedge 2 = 6$
 $6 = 6 \wedge 4$
 $= 2$

$\therefore 2$ is odd occurring

5) Even or odd using bitwise operator.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter your no: ";
    cin >> n;

    if (n & 1) {
        cout << "The no. is odd";
    }
    else {
        cout << "The no. is even";
    }
}
```

Dry Run:
n = 5

OP	Binary Calc	Result
n & 1	101 & 001	001
if()	non-0 is true	execute

∴ 5 is an odd no.

6) Factorial code.

```
#include <iostream>
using namespace std;
int main() {
    int n, fact = 1;
    cout << "Enter your no: ";
    cin >> n;

```

```
for (int i = 1; i <= n; i++) {
    fact = fact * i;
}
cout << fact;
}
```


* Stirling's Approximation:

→ #include <iostream>

#include <math>

using namespace std;

int main() {

int n;

double fact = 1;

cout << "Enter your no.:"

cin >> n;

fact = sqrt(2 * 3.14159 * n) * pow(n / 2.718, n);

cout << fact;

}

Prgrun

→ n = 5

∴ formula

$$2 \times 3.14159 \times 5 = 31.4159$$

$$\sqrt{31.4159} = 5.6049$$

$$5 / 2.71828 \approx 1.8394$$

$$(1.8394)^5 \approx 21.05$$

$$\text{fact} \approx 5.6049 \times 21.05$$

$$\text{fact} \approx 118.0$$

Dr
12/12

11/12/20

Experiment No. 3

1) Print the factorial, numbers from n to 1.

→ #include <iostream>

using namespace std;

void printNumbers(int n) {

if (n == 0)

return;

cout << n << " ";

printNumbers(n-1);

}

int main() {

int n;

cout << "Enter a no.:"

cin >> n;

printNumbers(n);

}

2) Sum of individual digits using recursion
→ #include <iostream>

using namespace std;
int sumDigits(int n) {
 if (n == 0)
 return 0;

return (n % 10) + sumDigits(n / 10);

}
int main() {

int n;
 cout << "Enter a no: ";
 cin >> n;

cout << "Sum of digits = " << sumDigits(n);

}

3) To find power m of the number N using recursion.
→ #include <iostream>

using namespace std;
int power(int n, int m) {
 if (m == 0)
 return 1;

return n * power(n, m - 1);

}
int main() {

int n, m;
 cout << "Enter number: ";
 cin >> n;
 cout << "Enter power: ";
 cin >> m;

cout << "Result: " << power(n, m);

}

4) Permutations of a string.
→ #include <iostream> #include <algorithm>

using namespace std;

```
void permute (string s, int index) {
    if (index == s.length()) {
        cout << s << endl;
    }
```

```
for (int i = index; i < s.length(); i++) {
```

```
    swap(s[index], s[i]);
    permute(s, index + 1);
}
```

```
int main(void) {
```

```
    string s;
```

```
    cin >> s;
```

```
    permute(s, 0);
}
```

5) To count the no. of subsets = sum
→ #include <iostream>

using namespace std;

```
int countSubsets(int i, vector<int> &arr, int sum) {
    if (sum == 0)
        return 1;
```

```
    if (i == arr.size()) || sum < 0
        return 0;
```

```
    int include = countSubsets(i + 1, arr, sum - arr[i]);
```

```
    int exclude = countSubsets(i + 1, arr, sum);
```

```
    return (include + exclude);
}
```

```
int main() {
```

```
    vector<int> arr = { 2, 3, 5, 6, 8, 10 };
```

```
    int sum = 10;
```

```
    cout << "Count of subsets: " << countSubsets(0, arr,
```

```
    sum);
}
```

palindrome & str length?

6) Palindrome:-

→ #include <iostream>

using namespace std;

bool isPal(string s, int l, int r) {

if (l >= r)

return true;

if (s[l] == s[r])

return false;

return isPal(s, l+1, r-1);

}
int main() {

string str;

cout << "Enter ";

cin >> str;

if (isPal(str, 0, str.length()-1))

cout << "Palindrome";

else

cout << "Not a palindrome";

}

7) Character count in a string

#include <iostream>

using namespace std;

int strlen(char str[], int i) {

if (str[i] == '\0')

return 0;

return 1 + strlen(str, i+1);

}
int main() {

char str[100];

cout << "Enter a string";

cin >> str;

int length = strlen(str, 0);

cout << "length = " << length;

}

Pr

17/3

- 1) Second largest Element in array.
→ #include <iostream>

using namespace std;

```
int main() {
    int n, a[100], temp;

    cout << "Enter size: ";
    cin >> n;
    cout << "Enter Array Elements: ";
cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j = i+1; j++) {
            if (a[i] > a[j]) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    cout << "Second largest = " << a[n-2];
    return
}
```

- 2) To check if an array is sorted or not.
→ #include <iostream>

using namespace std;

```
int main() {
    int n, a[100], flag = 1;
    cout << "Enter size: ";
    cin >> n;
    cout << "Enter elements: \n";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (int i = 0; i < n-1; i++) {
        if (a[i] > a[i+1]) {
            flag = 0;
            break;
        }
    }

    if (flag == 1) {
        cout << "Array is sorted.";
    }
    else {
        cout << "Array is not sorted.";
    }
}
```


3) Moving all zeroes to the end of the array.
→ #include <iostream>
using namespace std;

```
int main() {
    int n, a[100], i = 0, temp;
    cout << "Enter no. of elements: ";
    cin >> n;

    cout << "Enter array elements: ";
    for (i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (i = 0; i < n; i++) {
        if (a[i] == 0) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            j++;
        }
    }

    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
}
```

4) Rotating an array to left by one place.
→ #include <iostream>
using namespace std;

```
int main() {
    int n, a[100], temp;
    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    temp = a[0];
    for (int i = 0; i < n; i++) {
        a[i] = a[i+1];
    }

    a[n-1] = temp;
    cout << "After left rotation: ";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
}
```


5) Leaders problem in an array;
→ #include <iostream>

using namespace std;
int main() {

int n, a[5];

cout << "Enter elements:\n";

for (int i = 0; i < 5; i++) {

cin >> a[i];

}

cout << "Leaders are:\n";

for (int i = 0; i < 5; i++) {

int flag = 1;

for (int j = i + 1; j < 5; j++) {

if (a[i] < a[j]) {

flag = 0;

break;

}

}

if (flag == 1) {

cout << a[i] << " ";

}

6) Transpose of a matrix.
→ #include <iostream>

using namespace std;
int main() {

int r, c, a[10][10];

cout << "Enter rows and columns:";

cin >> r >> c;

cout << "Enter elements:\n";

for (int i = 0; i < r; i++) {

for (int j = 0; j < c; j++) {

cin >> a[i][j];

}

}

cout << "Transpose of matrix:\n";

for (int i = 0; i < c; i++) {

for (int j = 0; j < r; j++) {

cout << a[j][i] << " ";

}

cout << endl;

}

}

7) Averaging Matrix Elements in snake pattern.
→ #include <iostream>

```
using namespace std;
int main() {
    int a[3][3];
    cout << "Enter matrix elements:";
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cin >> a[i][j];
        }
    }
    cout << "Snake Pattern:\n";
    for (int i = 0; i < 3; i++) {
        if (i % 2 == 0) {
            for (int j = 0; j < 3; j++) {
                cout << a[i][j] << " ";
            }
        } else {
            for (int j = 2; j >= 0; j--) {
                cout << a[i][j] << " ";
            }
        }
        cout << endl;
    }
}
```

Or
14/13

16/3/20

Experiment No. 5

1) Division Method:-

→ #include <iostream>

```
using namespace std;
int main() {
    int key, m;
    cout << "Enter key:";
    cin >> key;
    cout << "Enter table size:";
    cin >> m;
    int hash = key % m;
    cout << "Hash Value = " << hash;
}
```

* Example:

Key = 1320

m = 11

$h(1320) = 1320 \% 11$

∴ Hash Value = 0

Value

2) Mid-Square Method:-
→ #include <iostream>

using namespace std;
int main() {
int key;

cout << "Enter Key:";
cin >> key;

int square = key * key;
int hash = (square / 10) % 100;

cout << "Hash Value = " << hash;

}

* Example:

key = 50

square = 50 × 50
= 2500

hash = (2500 / 10) % 100
= (250) % 100

hash = 50
value

3) Folding Method:-
→ #include <iostream>

using namespace std;
int main() {

string key; int sum = 0;
cout << "Enter Key:";
cin >> key;

for (int i = 0; i < key.length(); i++) {
if (i + 1 < key.length()) {
sum += (key[i] - '0') * 10 + (key[i + 1] - '0');
} else {
sum += key[i] - '0';

cout << "Hash Value = " << sum;

}

* Example: 54321

① i = 0

key[0] = '5', key[1] = '4'
∴ (5 * 10) + 4 = 50 + 4

sum = 54

② i = 2

key[2] = '3', key[3] = '2'
∴ (3 * 10) + 2 = 30 + 2

= 32

sum = 54 + 32 = 86

③ i = 4

key[4] = '1'

∴ now sum = key[i] - '0'
= 1

∴ sum = 86 + 1 = 87

4) Multiplication Method:-

→ #include <iostream>

using namespace std;

int main() {

int k, m;

float A = 0.35784

cout << "Enter Key ";

cin >> k;

cout << "Enter Table size";

cin >> m;

float x = k * A;

float frac = x - (int)x

int hash = m * frac;

cout << "Hash value = " << hash;

}

* Example:-

~~k = 1234~~

M = 100

A = 0.35784

So,

① $h(1234) = 1234 \times 0.35784$

$= 441.57456$

② Fractional part = 0.57456

Multiply by M $\therefore 100 \times 0.57456$

$= 57.456$

③ Take Floor

$\therefore \text{Hash Value} = \underline{57}$

* Course Structure Codes:

(i) #include <iostream>

#include <list>

using namespace std;

int main() {

int arr[] = {15, 11, 27, 8, 12};

int n = 5;

int size = 7;

list <int> hashtable[7];

for (int i = 0; i < n; i++) {

int index = arr[i] % size;

hashtable[index].push_back(arr[i]);

}

for (int i = 0; i < size; i++) {

cout << i << " -> ";

for (int x: hashtable[i]) {

cout << x << " ";

}

cout << endl;

}

}

O/P:

0 ->

1 -> 15 8

2 ->

3 ->

4 -> 11

5 -> 12

6 -> 27

Here,

→ separate chaining / open hashing is used, it is the most used collision resolving hashing technique in DS that uses linked list.

★ → Any 2 or more components that meet at the same point are chained together to form a single linked list known as a chain.

Dry Run:

Keys: 15, 11, 27, 8, 12

Therefore,

0

1 15, 8

2

3

4 11

5 12

6 27

As collision occurs at inserting keys 15, 8 they are added in the chain.

ii) Linear Probing in hashing

→ #include <iostream>

using namespace std;

int main() {

int arr[] = {12, 25, 35, 26, 25};

int n = 5;

int hashSize = 7;

int hashtable[7];

for (i = 0; i < hashSize; i++) {

hashTable[i] = -1; }

for (int i = 0; i < n; i++) {

int index = arr[i] % hashSize;

if (hashTable[index] == arr[i])

continue;

while (hashTable[index] != -1) {

index = (index + 1) % hashSize;

hashTable[index] = arr[i]; }

cout << "Hash Table: \n";

for (int i = 0; i < hashSize; i++) {

cout << i << " -> " << hashTable[i] << endl; }

}

O/P: → Hash Table

0 -> 35

1 -> -1

2 -> -1

3 -> -1

4 -> 25

5 -> 12

6 -> 26

* Linear Probing:-

- The formula for linear probing is

$$\text{index} = \text{key} \% \text{hashTable size}$$
- If the slot $\text{index} = [\text{hash}(n) \% T]$ is full then the next slot index is calculated by adding 1 ($\text{hash}(n+1) \% T$)

any Run

HashTableSize = 7

Keys = 12, 25, 35, 26, 25

Therefore,

Key

Hash

index

index (after linear probing)

12	12 % 7	5	5
25	25 % 7	4	4
35	35 % 7	0	0
26	26 % 7	5	6
25	25 % 7	4 (duplicate)	4

iii) Insert Members into a HashTable

→ #include <iostream>

using namespace std;

int main() {

int n, size;

cout << "Enter no. of elements:";

cin >> n;

int arr[n];

cout << "Enter " << n << " integers:\n";

for (i = 0; i < n; i++) {

cin >> arr[i];

}
cout << "Enter size of hash Table:";

cin >> size;

int hashTable;

for (int i = 0; i < size; i++) {

hashTable[i] = -1;

}

for (int i = 0; i < n; i++) {

int key = arr[i];

int index = key % size;

while (hashTable[index] != -1) {

index = (index + 1) % size;

}
hashTable[index] = key;

}

cout << "Hash Table:";

for (int i = 0; i < size; i++) {

if (hashTable[i] == 1)

cout << i << " -> " << hashTable[i] << endl;

else { cout << i << " -> EMPTY" << endl;

}
}

* O/P →

Enter no. of elements: 5
Enter 5 integers:

10

20

30

40

50

Enter size of hash Table: 6

Hash Table:

0 → 30

1 → EMPTY

2 → 20

3 → 50

4 → 10

5 → 40

Qr

14/4/26

Experiment No. 6

1) Iterative Binary Search:-
→ #include <iostream>

using namespace std;
int main() {

int i, mid, low, high, key, a[100], f; f=0;

cout << "Enter no. of elements:";
cin >> c;

cout << "Enter sorted array";
for (i=0; i<c; i++) {

cin >> a[i];
}

high = c-1;
low = 0;

cout << "Enter Key:";
cin >> key;

while (low <= high) {
mid = (low + high) / 2;

if (a[mid] == key) {

f = 1;
break; }

elseif (key > a[mid]) {

low = mid + 1; }

else {

high = mid - 1; }

}


```

if (f == 1) {
    cout << "Key Found";
}
else {
    cout << "Key not found";
}

```

O/P: → Enter no. of elements: 5

Enter sorted array:

2
3
4
5
6

Enter key: 7

Key not found.

2) Recursive Binary Search:-
→ #include <iostream>

using namespace std;

```

int search (int a[], int low, int high, int key)
{
    if (low > high) {
        return 0;
    }

```

```

    int mid = (low + high) / 2;
    if (a[mid] == key) {
        return 1;
    }

```

```

    elseif (key > a[mid]) {
        return search(a, mid+1, high, key);
    }

```

```

    else {
        return search(a, low, mid-1, key);
    }
}

```

int main () {

```

    int i, mid, key, high, low, a[100], c, key;
    cout << "Enter no. of elements:";

```

```

    cin >> c;
    cout << "Enter sorted array:";
    for (i = 0; i < c; i++) {
        cin >> a[i];
    }

```

```

    high = c - 1;
    low = 0;
    cout << "Enter Key:";

```

```

    cin >> key;
    if (search(a, low, high, key) == 1) {
        cout << "Key Found";
    }
}

```



```

else {
    cout << "key not found";
}
}

```

* O/P: →

Enter no. of elements: 5

Enter sorted array:

2

3

4

5

6

Enter key: 5

Key Found.

3) Recursive Binary Search (Tripled)

using namespace std;

```

int search (int a[], int low, int high, key, n) {
    if (low > high) {
        return 0;
    }
    int mid = (low + high) / 2;
    if (a[mid] == key) {
        cout << "\nkey found" << key;
        return search (a, 0, n-1, key, n);
    }
    else if (key > a[mid]) {
        return search (a, mid+1, high, key, n);
    }
    else {
        return search (a, low, mid-1, key, n);
    }
}
int main() {
    int a[] = {2, 3, 4, 5, 6, 7, 8, 9, 10};
    int n = sizeof(a) / sizeof(int);
    int low = 0, high = n-1, key;
    cout << "Enter no. of elements: ";
    cin >> n;
    cout << "Enter sorted array: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    cout << "Enter key: ";
    cin >> key;
    high = n-1;
    low = 0;
    search (a, low, high, key, n);
}

```


* O/P: →

Enter no. of elements: 5

Enter sorted array: 1

3

9

27

81

Enter key: 3

key found: 3

key found: 9

key found: 27

key found: 81

4) 1st occurrence of an element
→ #include <iostream>

using namespace std;
int main() {

int a[100], n, key, index;

cout << "Enter size: ";

cin >> n;

cout << "Enter sorted elements: ";

for (i = 0; i < n; i++) {

cin >> a[i]; }

cout << "Enter key: ";

cin >> key;

for (i = 0; i < n; i++) {

if (a[i] == key) {

index = i;

break;

}

cout << "1st occ of element is at index: " << index;

* O/P: -

Enter size: 4

Enter sorted elements:

1

2

2

3

Enter key: 2

1st occ of element is at index: 1

5) Last occurrence of an element

→ #include <iostream>
using namespace std;
int main() {

int a[100], n, key, index;
cout << "Enter size:";

cin >> n;

cout << "Enter sorted elements:";

for (i = 0; i < n; i++) {

cin >> a[i];

}

cout << "Enter key:";

cin >> key;

for (i = n; i > 0; i--) {

if (a[i] == key) {

index = i;

break;

}

}

cout << "Last occ of element at index << index;

}

* O/P: →

Enter size: 4

Enter sorted elements:

1

2

2

3

Last occ of element is at index 2

6) Peak element

→ #include <iostream>
using namespace std;
int main() {

int a[100], i, n;

cout << "Enter size:";

cin >> n;

cout << "Enter sorted elements:";

for (i = 0; i < n; i++) {

cin >> a[i];

}

for (i = 0; i < n; i++) {

if (a[i] > a[i-1] && a[i] > a[i+1]) {

cout << a[i] << " is a peak element\n";

}

* O/P: →

Enter size: 5

Enter sorted elements:

1

2

3

5

6

2 is a peak element

7) 2 Pointer Array

→ #include <iostream>

using namespace std;

int main()

{
int a[100], i, j, target, n;

cout << "Enter no. of elements:";

cin >> n;

cout << "Enter elements:";

for (i = 0; i < n; i++)

{
cin >> a[i];

}

i = 0;

j = n - 1;

cout << "Enter target:";

cin >> target;

while (i <= j)

{
if (a[i] + a[j] == target)

{
cout << "i = " << i << ", j = " << j << ", a[i] = " << a[i] << ", a[j] = " << a[j] << endl;

i++;

j--;

else if (a[i] + a[j] < target)

{
i++;

else if (a[i] + a[j] > target)

{
j--;

}

cout << "Enter no. of elements: 7

Enter elements: 1 2 3 4 5 6 8

Enter target: 10

8 2

4 6

5 5

16/4/26

Experiment 7

1.1) Sort function using STL (array)

#include <iostream>

#include <algorithm>

using namespace std;

int main()

{
int arr[10], n;

cout << "Enter size of array:";

cin >> n;

cout << "Enter elements:";

for (int i = 0; i < n; i++)

{
cin >> arr[i];

sort(arr, arr + n);

cout << "sorted array:";

for (int i = 0; i < n; i++)

{
cout << arr[i] << " ";

}

}

Enter size of the array: 5

Enter elements: 2 5 3 7 1

Sorted array: 1

2

3

5

7

1.2) Vector:-

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
```

```
vector <int> v;
```

```
int n, i;
```

```
cout << "Enter size: ";
```

```
cin >> n;
```

```
cout << "Enter elements: ";
```

```
for (i = 0; i < n; i++) {
```

```
int x;
```

```
cin >> x;
```

```
v.push_back(x);
```

```
}
```

```
sort(v.begin(), v.end());
```

```
cout << "Sorted vector: ";
```

```
for (i = 0; i < n; i++) {
```

```
cout << v[i] << " ";
```

```
}
```

```
}
```

* O/P: →

Enter size: 5

Enter elements: 5 3 1 2 4

Sorted vector: 1 2 3 4 5

2) Bubble Sort:-

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
```

```
int a[100], i, j, n;
```

```
cout << "Enter size: ";
```

```
cin >> n;
```

```
cout << "Enter elements: ";
```

```
for (i = 0; i < n; i++) {
```

```
}
```

```
for (j = 0; j < n; j++) {
```

```
if (a[j] > a[j+1]) {
```

```
swap(a[j], a[j+1]);
```

```
}
```

```
}
```

```
}
```

```
cout << "Sorted array: ";
```

```
for (i = 0; i < n; i++) {
```

```
cout << a[i] << " ";
```

```
}
```

```
}
```

* O/P: →

Enter size: 4

Enter elements: 7 5 8 3

Sorted array: 3 5 7 8

7

8